



Library Management System

Project Report

Developed by:

Shravan Jaiswal

Data Engineering Intern

Developed under Guidance of:

Mr. Prateek Khandelwal

10 July 2026

This document and the information contained herein are confidential to and the property of Systango. Unauthorized access, copying and replication are prohibited. This document must not be copied in whole or part by any means, without the written authorization of Systango.

Contents

1	Introduction	4
2	Database Design & Approach	4
2.1	Schema Design	4
2.2	Partitioning & Composite Primary Key	4
2.3	Fine Calculation Logic	4
2.4	Stored Procedures & Triggers Coordination	5
2.5	Dummy Data	5
3	EntityRelationship Diagram	6
4	Performance Optimization Techniques	7
4.1	Indexes	7
4.2	Partitioning	7
4.3	Tradeoffs	7
5	Query Execution Comparison	8
6	Conclusion	9

List of Figures

1	EntityRelationship Diagram of the Library Management System	6
---	---	---

List of Tables

1	Overdue Fine Rates	4
2	Created Indexes	7

1 Introduction

This project implements a full-featured Library Management System using MySQL 8.0. The system tracks authors, publishers, books, library members, and borrowing transactions. It includes stored procedures for borrowing and returning books, triggers to automatically update inventory and calculate fines, database partitioning for performance, and a comprehensive set of SQL queries to demonstrate proficiency in joins, subqueries, CTEs, window functions, and optimisation techniques.

2 Database Design & Approach

2.1 Schema Design

Six tables were created, normalised to 3NF to eliminate redundancy:

- **Authors** and **Publishers** simple lookup tables with autoincrement primary keys and descriptive attributes.
- **Books** references Authors and Publishers via foreign keys. Includes a CHECK constraint on `available_copies` ≥ 0 to prevent negative inventory.
- **Members** stores library member details. `email` has a UNIQUE constraint to prevent duplicate registrations.
- **Borrow_Transactions** the main transactional table. Contains issue/return dates and a calculated `fine` field. This table is **partitioned by** `YEAR(issue_date)` to enable efficient daterange queries as the table grows.
- **Deleted_Books** an audit table populated by a trigger whenever a book is deleted.

Important design decision: MySQL does not support foreign key constraints on partitioned tables (Error 1506). Therefore, `Borrow_Transactions` does not have physical foreign keys to `Members` or `Books`. Referential integrity is maintained at the application level through stored procedures, and the logical relationships are documented in the schema comments.

2.2 Partitioning & Composite Primary Key

To support partitioning by `YEAR(issue_date)`, MySQL requires the partition column to be part of the primary key. We used a **composite primary key** on `(transaction_id, issue_date)`. `transaction_id` is still unique via autoincrement, so data integrity is preserved. The table is partitioned into yearly ranges (2023–2026) plus a MAXVALUE partition for future data.

2.3 Fine Calculation Logic

Fines are computed only at return time (not daily), using a **progressive tiered rate** based on the book's price to discourage late returns:

Table 1: Overdue Fine Rates

Overdue Days	Daily Fine Rate
1–7	5% of book price
8–14	10% of book price
15–21	15% of book price
22+	20% of book price

The total fine is capped at the book's price, so a member never pays more than the replacement value. This logic is implemented in **Trigger 2** (AFTER UPDATE ON `return_date`).

2.4 Stored Procedures & Triggers Coordination

To avoid duplication of logic, stored procedures focus on core actions, while triggers handle automatic side effects:

- **BorrowBook** procedure inserts a new transaction. **Trigger 1** (AFTER INSERT) automatically decrements `available_copies`.
- **ReturnBook** procedure updates `return_date`. **Trigger 2** (AFTER UPDATE) calculates the fine (if overdue) and increments `available_copies`.
- **Trigger 3** (AFTER DELETE ON `Books`) writes the deleted record to the `Deleted_Books` audit table.

This design ensures data integrity even if transactions are inserted/updated outside the procedures (e.g., bulk loads).

2.5 Dummy Data

500 rows per table were generated using WITH RECURSIVE CTEs with random functions. Books span categories (Fiction, NonFiction, Science, History), prices 100–2000, and borrow transactions cover 2023–2026. Approximately 70% of books have been returned (some overdue), while 30% are still borrowed.

3 EntityRelationship Diagram

The ER diagram of the final database is shown below. It illustrates the tables, their columns, primary keys, foreign keys, and the relationships among them.

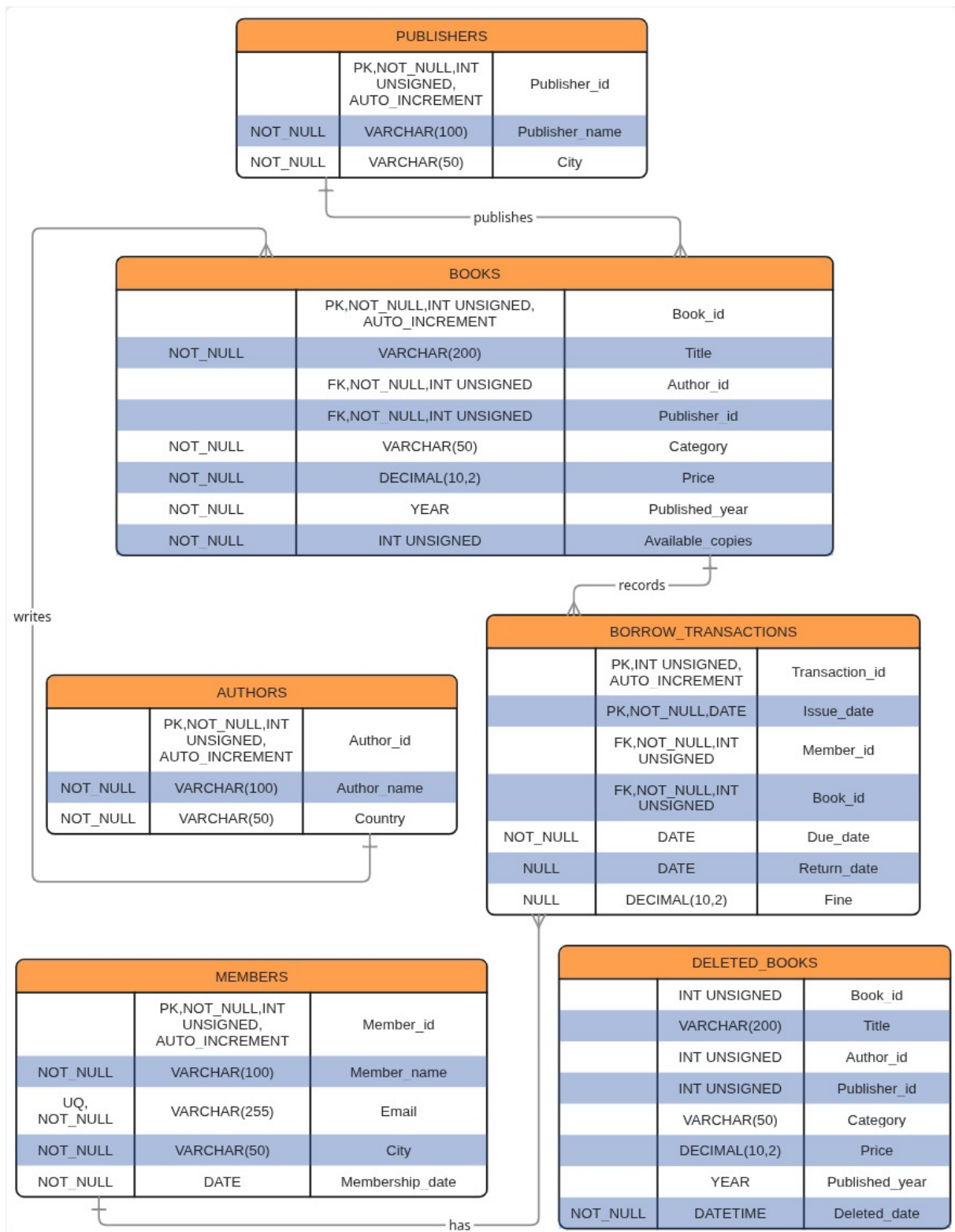


Figure 1: EntityRelationship Diagram of the Library Management System

4 Performance Optimization Techniques

4.1 Indexes

Three indexes were created on the `Borrow_Transactions` table to speed up frequent query patterns:

Table 2: Created Indexes

Index Name	Column	Purpose
<code>idx_member_id</code>	<code>member_id</code>	Accelerates joins between <code>Members</code> and <code>Borrow_Transactions</code> , and filters on a specific members borrow history.
<code>idx_book_id</code>	<code>book_id</code>	Speeds up joins with <code>Books</code> , lookups for a books borrowing history, and availability checks.
<code>idx_issue_date</code>	<code>issue_date</code>	Optimises daterange queries (e.g., borrows in 2024) and works together with partitioning.

Without these indexes, queries filtering on `member_id` or `book_id` would cause a full partition or table scan. With the indexes, the database can locate rows directly, dramatically reducing the number of rows examined and the response time.

4.2 Partitioning

`Borrow_Transactions` is partitioned by `YEAR(issue_date)` using `RANGE` partitioning. Partitions were created for 2023, 2024, 2025, 2026, and a `MAXVALUE` partition for future data. This enables **partition pruning**: when a query filters on `issue_date`, MySQL reads only the relevant partitions, not the entire table. Partitioning also simplifies data management (e.g., archiving old years by dropping a partition instead of running expensive `DELETE` statements).

4.3 Tradeoffs

- **Foreign keys** could not be enforced on the partitioned table. We mitigated this by using stored procedures for all data modifications and documenting the logical relationships.
- The **composite primary key** (`transaction_id`, `issue_date`) is slightly wider than a singlecolumn key, but this is the required design for MySQL partitioning and has negligible storage impact.

5 Query Execution Comparison

To quantify the performance improvement, we ran a representative query before and after creating the indexes. The query selects borrow transactions for a specific member in a given year:

```
1 SELECT * FROM Borrow_Transactions
2 WHERE member_id = 42 AND issue_date >= '2024-01-01';
```

Before indexing

(indexes dropped, only partitioning active):

```
-> Filter: ((member_id = 42) and (issue_date >= DATE'2024-01-01'))
    (cost=34.3 rows=11.1) (actual time=0.231..0.316 rows=1 loops=1)
    -> Table scan on Borrow_Transactions
        (cost=34.3 rows=333) (actual time=0.11..0.272 rows=333 loops=1)
```

After indexing

(all three indexes present):

```
-> Index lookup on Borrow_Transactions using idx_member_id (member_id=42),
    with index condition: (issue_date >= DATE'2024-01-01')
    (cost=0.283 rows=1) (actual time=0.035..0.039 rows=1 loops=1)
```

Key observations

- Without indexes, the database performed a **full partition scan**, examining all 333 rows in the relevant partition. Actual time was 0.23–0.32 ms.
- With the `idx_member_id` index, the database performed an **index lookup**, reading only 1 row. Actual time dropped to 0.035–0.039 ms.
- The number of rows examined fell from **333 to 1** a 99.7% reduction.
- Even on this small 500row table, the benefit is measurable; on a larger production dataset, the performance gain would be far more dramatic.

This comparison confirms that the indexes are effective and that the database is welltuned for the expected query patterns.

6 Conclusion

The Library Management System demonstrates a complete SQL project suitable for a data engineering context. Key achievements include:

- A normalised, wellconstrained schema with logical and physical integrity.
- Partitioning and indexing strategies that improve query performance and data manageability.
- A progressive fine system that balances deterrence and fairness.
- Modular stored procedures and triggers that automate core library operations while maintaining data consistency.
- A rich set of queries covering joins, subqueries, CTEs, and window functions, all tested against a synthetic dataset of 500 records.

The project is ready for extension: additional features like book reservations, waitlists, or reports could be built on this foundation, and the table design would support them without fundamental changes.